

Hiring Automation Workflow Walkthrough

Introduction

Hello, today I will explain this hiring automation workflow in a simple and practical way.

This workflow is built in n8n. It automates the hiring process from application intake to resume screening, candidate database updates, interview scheduling, and email communication.

The workflow supports two roles:

- SWE, which means Software Engineer
- BDM, which means Business Development Manager

Both roles are handled inside one workflow, but the screening logic is different for each role.

Overall Architecture

The workflow architecture is divided into these main parts:

1. Source sheets
2. Data cleaning and normalization
3. Master candidate database
4. Resume download and text extraction
5. AI screening
6. Decision logic
7. Calendar and email automation
8. Final sheet updates

The source sheets contain raw applications. The master sheet works as the main database. AI is used only after the candidate data is cleaned and verified. After AI screening, the workflow decides what action should happen next.

Key Assumptions

This workflow is designed with a few important assumptions.

All required application fields are treated as compulsory. A candidate must provide their name, email, role-related details, and resume link.

The resume must be uploaded as a PDF file. The workflow is designed to process PDF resumes only.

If the resume is missing, the link is invalid, or the uploaded file is not a proper PDF resume, the candidate should not move forward automatically.

This assumption keeps the workflow reliable because the AI screening depends on clean candidate data and readable resume text.

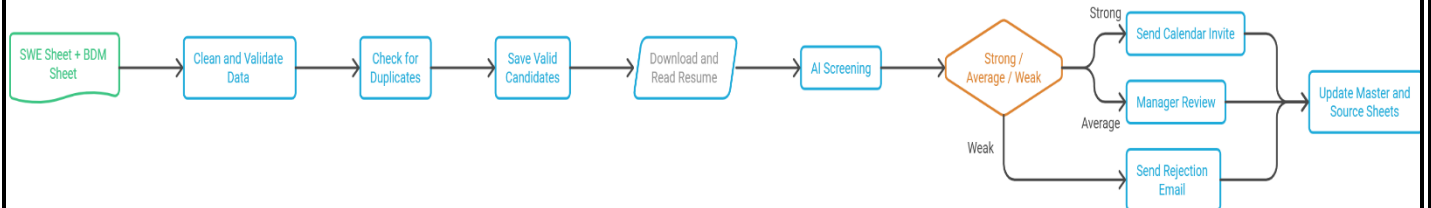
The basic flow is:

Step 1: Reading Applications

The workflow starts by reading candidate applications from two Google Sheets.

One sheet is for SWE candidates.

Candidate Screening and Processing Flow



The second sheet is for BDM candidates.

After reading the data, the workflow adds a role value to each candidate.

For SWE applications, it adds:

```
role = SWE
```

For BDM applications, it adds:

```
role = BDM
```

This role is important because later the AI uses role-specific rules.

Step 2: Data Cleaning

Raw form data is not always clean. Different forms can have different column names, empty values, extra spaces, or invalid resume links.

The workflow cleans the data before processing.

It removes invalid empty values such as:

```
empty string
nan
none
-
n/a
null
```

It also trims extra spaces from values.

For example:

```
" Alex Morgan " becomes "Alex Morgan"
```

The workflow searches for important fields using exact field names and fallback keywords.

For SWE candidates, it looks for:

- full name
- email address
- resume or CV
- programming languages or skills
- GitHub or portfolio link

For BDM candidates, it looks for:

- name
- contact email
- resume
- sales experience
- LinkedIn profile

This is helpful because different forms may use slightly different column names.

Step 3: Resume Link Cleaning

The workflow extracts the Google Drive file ID from the resume link.

It supports different Drive URL formats, such as:

```
https://drive.google.com/file/d/FILE_ID/view  
https://drive.google.com/open?id=FILE_ID  
https://drive.google.com/uc?id=FILE_ID
```

If the workflow cannot find a valid Drive file ID, the candidate is skipped.

This protects the workflow from failing later during resume download.

Step 4: Required Field Validation

Before saving or screening a candidate, the workflow checks required fields.

Required fields are:

- name
- email
- resume URL
- valid Drive file ID
- PDF resume file

If any required field is missing, the candidate is not processed.

For example:

```
Candidate has name and email but no resume link  
→ workflow skips the candidate
```

Another example:

```
Candidate uploads a non-PDF resume  
→ workflow treats it as invalid and does not process it automatically
```

The skipped reason is recorded, so the team can understand why the candidate was not processed.

Step 5: Candidate ID Generation

Every valid candidate gets a unique Candidate ID.

The Candidate ID is generated using the candidate email and role.

The logic is:

```
email + role → hash → Candidate ID
```

For example:

```
alex@example.com + SWE
```

The workflow converts the email to lowercase and the role to uppercase.

Then it creates a SHA-256 hash from this combination.

Only the first 12 characters of the hash are used.

The final Candidate ID format looks like:

```
SWE-7A9F2C31B8D4  
BDM-91AC8F20E112
```

This makes the Candidate ID stable.

If the same person applies again for the same role with the same email, the generated Candidate ID will be the same.

Step 6: Duplicate Prevention

The workflow avoids duplicate candidates by checking the master sheet.

It uses this key:

```
email + role
```

For example:

```
alex@example.com | SWE
```

If this key already exists in the master sheet, the workflow checks the candidate status.

If the candidate already has a final status, the workflow skips them.

Examples of final statuses:

- screened
- interview_scheduled
- rejected

- manual_review_notified

This prevents duplicate AI screening, duplicate emails, and duplicate calendar events.

The workflow also checks duplicates inside the same run.

So if the same candidate appears twice in one sheet run, only the first valid record is processed.

Step 7: Retry Protection

The workflow does not permanently block candidates if an earlier run failed.

Some statuses are retryable.

Retryable statuses include:

```
pending  
screening_error  
resume_error
```

If a candidate already exists with one of these statuses, the workflow allows processing again.

This is useful if the resume failed to download, AI returned invalid output, or the workflow stopped halfway.

So the logic is protected but still practical.

Step 8: Master Candidate Sheet

The master sheet is the central database.

It stores:

- Candidate ID
- name
- email
- role
- experience
- resume URL
- Drive file ID
- AI score
- AI classification
- AI summary
- strengths
- concerns
- interview recommendation
- contacted status

- calendar event link
- workflow status
- error note

This makes the hiring process easier to audit because every candidate has one main record.

Step 9: Resume Download and Text Extraction

After the candidate is saved, the workflow downloads the resume from Google Drive.

The expected resume format is PDF.

Then it extracts the text from the PDF resume.

This extracted text is sent to the AI model.

If the resume is empty, unreadable, corrupted, not a PDF, or not actually a resume, the candidate should not move forward automatically.

If resume text still reaches the AI but the content is unusable, the AI is instructed to score it low and mark it as weak.

This protects the system from giving high scores to candidates without real evidence.

Step 10: AI Screening Logic

The AI is instructed to be strict, fair, and evidence-based.

It can only use:

- resume text
- candidate metadata
- role information

The AI is told not to:

- browse the internet
- invent facts
- assume missing experience
- reward vague claims
- change scoring randomly

The AI must return a JSON result with:

- score
- classification
- summary
- key strengths

- concerns
- interview recommendation

Step 11: Score and Classification Rules

The score range is 0 to 100.

The classification is based on score:

```
75 to 100 = strong
50 to 74 = average
0 to 49 = weak
```

The workflow also checks the AI response after it comes back.

If AI returns a wrong classification, the workflow fixes it based on the score.

For example:

```
AI score = 82
AI classification = average
```

The workflow changes classification to:

```
strong
```

This keeps the result consistent.

Step 12: Role-Based Evaluation

For SWE candidates, the AI checks engineering evidence.

It looks at:

- production software experience
- programming languages
- frontend or backend work
- APIs and databases
- system design
- cloud or DevOps
- testing and debugging
- GitHub or portfolio
- measurable impact

For BDM candidates, the AI checks business and sales evidence.

It looks at:

- sales experience
- revenue ownership

- quota or target achievement
- lead generation
- pipeline management
- client handling
- negotiation
- CRM usage
- account growth

This makes the screening realistic because SWE and BDM candidates need different skills.

Step 13: If Logic After Screening

After AI screening, the workflow uses if/switch logic.

The decision is based on:

```
ai_classification
```

If:

```
ai_classification = strong
```

Then:

```
schedule interview  
send interview email  
update status as interview_scheduled
```

If:

```
ai_classification = average
```

Then:

```
send manual review email to hiring manager  
update status as manual_review_notified
```

If:

```
ai_classification = weak
```

Then:

```
send rejection email  
update status as rejected
```

This is the main decision engine of the workflow.

Step 14: Interview Scheduling Logic

Only strong candidates are scheduled for interviews.

The interview window is:


```
2:00 PM to 4:00 PM Pakistan time
```

The workflow creates four 30-minute slots:

```
2:00 PM to 2:30 PM
2:30 PM to 3:00 PM
3:00 PM to 3:30 PM
3:30 PM to 4:00 PM
```

Only four strong candidates are scheduled per day.

The workflow uses the loop index to decide the slot.

The logic is:

```
slot number = candidate index % 4
business day offset = candidate index / 4
```

Example for 10 strong candidates:

```
Candidate 1 → Day 1, 2:00 PM
Candidate 2 → Day 1, 2:30 PM
Candidate 3 → Day 1, 3:00 PM
Candidate 4 → Day 1, 3:30 PM
Candidate 5 → Day 2, 2:00 PM
Candidate 6 → Day 2, 2:30 PM
Candidate 7 → Day 2, 3:00 PM
Candidate 8 → Day 2, 3:30 PM
Candidate 9 → Day 3, 2:00 PM
Candidate 10 → Day 3, 2:30 PM
```

The workflow only schedules from Monday to Friday.

If the next day is Saturday or Sunday, it skips to Monday.

The timezone is fixed to:

```
Asia/Karachi
```

This prevents the interview time from shifting to 6 PM because of server timezone conversion.

Step 15: Calendar Invite Logic

The Google Calendar node creates the interview event.

The candidate email is added as an attendee:

```
attendee = candidate email
```

The workflow also sets:

```
sendUpdates = all
```

This tells Google Calendar to send the event invite to the candidate.

So for strong candidates, two things happen:

1. Google Calendar invite is sent
2. Gmail interview confirmation email is sent

Step 16: Interview Email Logic

After the calendar event is created, the workflow builds the interview email.

The calendar node returns event data, such as:

- event start time
- calendar event link
- meeting or event details

The email builder uses this output and creates the email body.

The Gmail node sends:

```
{{ $json.output.email_body }}
```

This means Gmail is sending the email body prepared by the previous node.

The recipient is:

```
{{ $json.output.candidate_email }}
```

So the email goes to the same candidate who was scheduled for the interview.

Step 17: Average Candidate Flow

Average candidates are not rejected automatically.

Instead, the workflow sends a manual review email to the hiring manager.

That email includes:

- candidate name
- role
- email
- AI score
- classification
- summary
- strengths
- concerns

This allows a human to make the final decision.

Step 18: Weak Candidate Flow

Weak candidates receive a rejection email.

The rejection email is polite and professional.

It thanks the candidate for applying and tells them that the company is not moving forward at this time.

This keeps the candidate experience respectful.

Step 19: Source Sheet Updates

After processing, the workflow updates the original source sheet.

It marks the source row as processed.

This is important because it prevents the same row from being picked again in the next workflow run.

The workflow uses the role to decide which source sheet should be updated.

If role is SWE, it updates the SWE source sheet.

If role is BDM, it updates the BDM source sheet.

Step 20: Final Master Sheet Updates

The workflow also updates the master sheet after communication is completed.

For strong candidates:

```
contacted = YES
status = interview_scheduled
calendar_event_link = saved
```

For average candidates:

```
contacted = YES
status = manual_review_notified
```

For weak candidates:

```
contacted = YES
status = rejected
```

This gives the hiring team a clear final status for every candidate.

Step 21: Protection and Reliability

The workflow is protected in multiple ways.

It protects against missing data by checking required fields.

It protects against duplicate candidates by checking email and role.

It protects against duplicate processing by checking processed status.

It protects against broken resume links by validating the Drive file ID.

It protects against unsupported resume formats by assuming PDF resumes only.

It protects against bad AI output by parsing and validating the AI JSON.

It protects against wrong classification by forcing the classification to match the score.

It protects against weekend scheduling by only allowing Monday to Friday interview dates.

It protects against timezone errors by using Asia/Karachi timezone.

It protects against missing calendar invites by adding the candidate as attendee and setting sendUpdates to all.

Step 22: Simple Example

Suppose 10 strong candidates are found in one run.

The workflow will not schedule all 10 in one day.

It will schedule only 4 per day:

Friday:

2:00 PM

2:30 PM

3:00 PM

3:30 PM

Monday:

2:00 PM

2:30 PM

3:00 PM

3:30 PM

Tuesday:

2:00 PM

2:30 PM

This is more realistic and manageable for the hiring team.

Conclusion

In simple words, this workflow automates the hiring process with proper checks and decision logic.

It reads candidates, cleans data, avoids duplicates, generates Candidate IDs, downloads resumes, screens candidates with AI, and sends the correct communication.

Strong candidates get interview invites.

Average candidates go to manual review.

Weak candidates get rejection emails.

The workflow is designed to be realistic, controlled, and protected from common automation errors.